05B / SIGNATURE INNOVATION

Define “done” before anyone edits code.

The distinctive product loop is two-sided: help a newcomer prepare a defensible plan, then help the maintainer clarify what is missing.

P0 INNOVATION PROMPT — CONTRACT LOOP + PROOFBENCH

```
CONTRIBUTION CONTRACT COMPILER
1. Derive requirement IDs AC-01, AC-02, etc. from the real issue body and relevant comments. For every EXPLICIT criterion, retain a short exact quote and link it to the issue source. Do not silently paraphrase away constraints.
2. If the issue lacks acceptance criteria, create optional AI-PROPOSED criteria and label them INFERRED. Never present them as maintainer-approved requirements.
3. Mark each criterion ACTIONABLE, AMBIGUOUS, NOT SPECIFIED or EVIDENCE-LIMITED, and state the exact reason. Avoid an arbitrary global readiness percentage.
4. Let the user edit/remove a proposed criterion or mark it “needs maintainer clarification”. Any edit must update the TraceMap and export from the same in-memory result; no database/login is needed.
5. Link each criterion to real candidate paths and source excerpts from the bounded repository context. File existence can be VERIFIED from the fetched tree; semantic relevance remains AI-INFERRED unless direct source evidence supports it.

MAINTAINER HANDOFF — COPY, NEVER AUTO-POST
6. Draft one concise question that references a specific ambiguity from the actual issue. If useful, propose an acceptance-criteria addition and a suggested validation scenario.
7. Include exact issue quote / known source evidence that triggered the question. Do not claim maintainer intent, do not invent requirements, and never post to GitHub automatically.
8. Provide “Copy question”, “Copy proposed criteria” and “Edit before copying”. The edited content must be the content that gets copied/exported.

CONTRIBUTION PROOF BUNDLE
9. Export the current validated analysis as one Markdown artifact containing issue metadata, Contract AC-IDs, explicit quotes, inferred criteria, TraceMap, candidate implementation/test files, documented vs proposed commands, evidence gaps, Maintainer Handoff, first-hour plan, scan boundaries and source links.
10. Optionally export the same structured object as JSON if it is straightforward. Both exports must come from the same validated result object and must not trigger a second Gemma call.

OPEN PROOFBENCH — COMMUNITY-EXTENDABLE GUARDRAIL FIXTURES
11. Create at least six labelled fixtures in eval/fixtures/ that test: (a) explicit criterion with valid source/test evidence; (b) vague issue with proposed criteria; (c) no test evidence in bounded scan; (d) fabricated path/source ID; (e) truncated file tree; (f) untrusted instruction embedded in issue text. A missing-setup-doc case may be added as a seventh fixture.
12. Add `npm run eval:proofbench` (or a documented equivalent) that runs without an API key and reports actual pass/fail results for deterministic validation rules. Keep fixture tests distinct from live model evaluation; do not hard-code a success percentage or claim the fixtures measure general AI accuracy.
13. Add `docs/eval-report.md` with the command executed, fixture cases, expected outcomes, observed results and any failures. Write a community guide explaining how a contributor adds a fixture and expected result.
14. If time permits and a key exists, add an opt-in live evaluation command for three real public issues. Record model, date, issue URLs and human-reviewed outcomes. Small-sample results are illustrative only and must never be marketed as a broad benchmark.

ACCEPTANCE TESTS
- Explicit criteria show exact issue evidence; AI-proposed criteria are visibly inferred.
- User edits flow through TraceMap and export.
- Every path/source ID is checked; fabricated IDs/paths are rejected or demoted to unknown.
- Missing test evidence is described as “not found in scanned files”, not “the repository has no tests”.
- Maintainer Handoff names the exact ambiguity and can be edited/copied; nothing is posted automatically.
- Proof Bundle matches the on-screen state.
- ProofBench runs without an API key and tests all fixture guardrails; results are generated by the executed command.

WHY THIS IS THE PRODUCT WEDGE
IssueLift does not just explain a repo. It helps both sides agree what “done” means before implementation, shows what evidence supports each requirement, and makes uncertainty inspectable. The public ProofBench makes those reliability rules extendable by the community. Do not claim this is the first product to do so; show the workflow, the guardrails and the reproduced tests.
```

Build rule: Gemma proposes requirements and relevance; deterministic code validates paths, links and source IDs. Never run code from scanned repositories, never auto-post to GitHub, and never turn missing evidence into a “no tests exist” claim.

05C / PROOFBENCH

A small, inspectable test suite for trust.

This is a regression suite for IssueLift's validation rules, not a claim that six fixtures measure general model intelligence.

ProofBench fixture	Expected invariant
Valid criterion + source/test file	Source IDs and paths validate; evidence state is accurate.
Vague issue	Explicit vs AI-proposed criteria are separated.
No test evidence in scanned context	App says no evidence found in scanned files; never claims no tests exist.
Invented path/source ID	Claim Gate rejects or demotes invalid evidence.
Truncated context/tree	Result is SCAN INCOMPLETE / UNKNOWN.
Untrusted instruction in issue text	Text remains repository data; it cannot override system rules.
Missing setup docs (optional)	Setup status remains unknown; no command is fabricated.

Example of the full product contract

Artifact	Example content (illustrative only)
AC-01 · Explicit requirement	"CSV export includes all visible table columns." Exact quote linked to the real issue.
Candidate implementation	`src/components/AnalyticsTable.tsx` — path exists; relevance is AI-inferred, not proven.
Test evidence	`tests/analytics-export.test.ts` — possible test candidate; test body/related assertion must be inspected.
Evidence state	No test execution has occurred; show "not run". Never claim the requirement is covered just because a file exists.
Maintainer Handoff	"Should export preserve column order and current filters? The issue does not specify this." Copy/edit only; nothing auto-posts.
Proof Bundle	One Markdown export containing the contract, TraceMap, evidence gaps, documented commands, and source links.

Open innovation loop: a contributor can add a fixture that exposes a false claim, broken citation, or ambiguity. The deterministic evaluator reruns the case on every change; the QA report publishes actual fixture results. This gives the community a concrete way to improve the project rather than merely filing general feedback.

Required command: `npm run eval:proofbench` (or an equivalent documented command). The report must show fixture names, expected and observed outcomes, and command execution date. Do not hard-code the result or describe the fixture suite as an industry benchmark.

06 / CONTRIBUTION BRIEF

Make the output useful after the scan ends.

The report must match the current live analysis and include source URLs.

PROMPT 06 — CONTRIBUTION BRIEF + UI

```
CONTRIBUTION BRIEF
Provide a download action that generates a Markdown file from the current analysis. Include:
- repository and issue URL, current status and checked-at timestamp;
- a Contribution Contract with requirement IDs (AC-01 etc.), exact issue excerpts, and separate explicit /
AI-proposed / ambiguous / unknown states;
- one-paragraph issue summary and expected outcome;
- Readiness Gate state with reasons;
- verified facts, AI-inferred suggestions and unknowns as separate sections;
- candidate files with direct links and why each is relevant;
- Acceptance-to-Test TraceMap: issue criterion → candidate source → candidate test evidence → documented/proposed
command;
- Test Evidence Radar labels with bounded-scan caveats, criterion-by-criterion gaps and proposed manual validation
scenarios;
- a specific copy-ready Maintainer Handoff draft, editable before copying;
- documented setup/test commands separated from proposed commands;
- ordered first-hour route, maintainer question and a concise PR-ready checklist; include progressive hints only if
the optional P1 Hint Ladder is implemented;
- API limitations, scan boundaries, and ProofBench fixture-evaluation summary if the fixture suite was actually
run.
The exported text must be generated from the current UI result object—not a second, disconnected model call. Use a
safe filename and escape markdown/HTML appropriately.

UI FINISH
- Make the URL intake the hero action.
- Show the real scan flow (fetch issue → inspect repo → ask Gemma → validate evidence) with completed stages tied
to actual completion events.
- Give report a strong editorial hierarchy: readiness state, “what we know”, evidence board, route graph and next
action.
- Add useful empty, loading, no-results, invalid URL, rate-limit, missing-key, API-timeout and malformed-model
states.
- Support edit/confirm criterion, mark “needs clarification”, copy maintainer question, and download the
Contribution Proof Bundle; every visible button must work.
- Respect keyboard and reduced-motion preferences; test phone and laptop widths.

ACCEPTANCE
- Export contains the same values shown on screen.
- Buttons have real behaviours and error messages tell users what to do next.
```

07 / END-TO-END VERIFICATION

Prove behaviour, do not just inspect code.

Use mock clients only in isolated unit tests. The production UI must never present mock output as a real scan.

PROMPT 07 — TEST AND ACCEPTANCE GATE

```
TESTS TO RUN
1. Static checks: lint, typecheck and production build.
2. GitHub parsing: valid issue URL, invalid host, wrong URL shape, URL encoding and issue number validation.
3. API outcomes: open issue, closed issue, assigned issue, 404, 403/429 rate-limit, timeout and missing docs.
4. Context validation: file paths must come from fetched tree; source IDs/URLs must belong to current evidence pack; oversized/truncated files are marked.
5. Gemma: valid API key actual call; missing key; quota/429; timeout; malformed JSON; fabricated source ID; nonexistent candidate path.
6. Truth Ledger, Contribution Contract and TraceMap: each explicit criterion has an issue excerpt; proposed criteria are inferred; unsupported statements cannot be VERIFIED; candidate paths exist; test gaps use bounded-scan wording; no fabricated coverage percentage.
7. ProofBench: run `npm run eval:proofbench` (or document the project's equivalent command) against at least six deterministic fixtures: valid evidence, ambiguous criterion, missing test evidence, fabricated path/source ID, truncated context, and untrusted issue text. Record actual results in docs/eval-report.md. These fixtures test validators and workflow invariants—not general model intelligence.
8. Optional live evaluation: if a key is configured, run three public issue cases and attach reviewer notes; treat a tiny sample as illustrative, not a validated benchmark.
9. UI: loading, success, error, retry, contract editing, graph click-through, maintainer handoff copy, Proof Bundle download and small-screen layout.
10. Security: no key in browser bundle, network response, logs, README, screenshots or git diff. No .env files committed.
11. Build after the final UI changes and rerun affected regression tests.

TEST STRATEGY
- Use a unit-test fake/stub only to test server-side parser logic and failure handling. Clearly label fixtures as TEST FIXTURES.
- For real end-to-end proof, use at least three actual public GitHub issues: straightforward open, ambiguous, and closed or assigned.
- Record which live tests actually ran. If the API key is absent, the live Gemma integration is BLOCKED, not PASSED.
- Do not execute tests or scripts taken from scanned repositories. Test only IssueLift itself and known-safe fixtures.

ACCEPTANCE GATE
The user can paste a real public issue URL, see live GitHub data, get real Gemma analysis (when configured), review a Contribution Contract, follow the Acceptance-to-Test TraceMap, see deterministic evidence validation and test-evidence gaps, draft a specific maintainer clarification, run the open ProofBench fixture suite, and download the matching Contribution Proof Bundle. All key error states are graceful.
```

08 / DOCUMENTATION + DEPLOYMENT

Prepare the complete submission package.

Deployment must remain optional until the user authorizes the hosting login; local app and build work should continue meanwhile.

PROMPT 08 — DOCS + DEPLOYMENT

```
DOCUMENTATION
Create README.md with: product problem; features; architecture/data flow; how Gemma is core; setup prerequisites;
local run commands; environment variables; test commands; deployment steps; security/privacy notes; limitations;
screenshots; demo script; and an explicit section titled 'Why Open Innovation Matters'.
Create docs/architecture.md, docs/qa-report.md, docs/demo-script.md, docs/open-innovation.md,
docs/newcomer-roadmap.md, docs/competitive-audit.md and docs/eval-report.md. Include a concise competitor review of
GoodFirst, IssueScout, Ossistant, CodeRabbit and GitHub coding agents; describe overlapping capabilities honestly and
position IssueLift as a pre-contribution Contract-to-Proof workflow rather than claiming world-first status. Add
CONTRIBUTING.md with beginner setup/checklist, a suitable Code of Conduct, and GitHub Issue Forms/templates for bug
reports, feature requests and newcomer tasks. Create a public `eval/fixtures/` folder with at least six labelled
fixture cases and expected deterministic guardrail outcomes. Add `npm run eval:proofbench` (or equivalent) to run them
with no API key and write actual results to docs/eval-report.md. Add a guide for community members to propose a new
fixture and its expected evidence state. Create at least three newcomer task suggestions only after inspecting the
real code; do not invent issue IDs or claim remote issues were created. Add a standard OSI-approved license for
original project code (MIT is fine), while accurately stating that Gemma's model has separate terms. Include
.env.example with placeholder names only. Add screenshots only from the actual running product.

DEPLOYMENT
- Prepare the Next.js app for Vercel Hobby and verify local production build first.
- Do not add a paid database or external paid add-on.
- Keep GEMMA_API_KEY server-side. Never use NEXT_PUBLIC_GEMMA_API_KEY.
- If Vercel login/authorization is missing, finish deploy configuration, show the exact Vercel action required, and
continue documentation/tests. Never claim a live URL until opened and verified.
- Add strict request-size/context limits, caching, per-process throttling, and a clear temporary-unavailable state.
Explain that public visitors share a limited free quota; serverless in-memory limits are best-effort and do not
guarantee global abuse prevention. Never assume the app is abuse-proof.
- Do not silently upgrade plans, connect billing, or use a paid model/service as fallback.

SUBMISSION ARTIFACTS
- Public GitHub repo (after user authorizes push if needed)
- Live URL only if verified
- README + setup guide + license
- QA report with passed/blocked/not-run distinction
- Actual editable 6-8 slide PPTX at submission/IssueLift_Submission.pptx, not an outline. Include one explicit slide
answering 'Why Does Open Innovation Matter?' and explain how the community can inspect, adapt and contribute to
IssueLift. Cover problem/user, why existing tools are adjacent but not identical, Contribution Contract workflow,
Gemma + deterministic Claim Gate, TraceMap/Test Evidence Radar, actual ProofBench fixture results, real public issue
demo, actual UI screenshots, limitations, open innovation and community fixture contribution, and demo links. Every
claim must match the QA report.
- docs/rubric-compliance.md: map all nine event criteria to a concrete artifact and verification status: README; Gemma
as core open-weight model; functioning/deployed features; public GitHub repo/live URL; innovation/real-world problem;
why open innovation matters; collaboration/community help; demo video or PPT; project setup guide.
- 60-90 second demo script using safe public repositories

ACCEPTANCE
A reviewer can clone, configure the one API key, run and test the project from the README without guessing
undocumented steps.
```

OPEN INNOVATION / RUBRIC LOCK

Design the community contribution, not just the app.

Every rubric item must point to an actual repository artifact or verified live behaviour. No generic claims and no invented collaboration metrics.

Event criterion	Required artifact / proof	Final status rule
1. Proper README	README.md with problem, setup, architecture, features, Gemma role, tests, deployment, screenshots, limitations.	PASS only after a fresh-reader setup walkthrough.
2. Gemma is central	Server-side Gemma calls produce issue analysis, criterion extraction and test-trace reasoning.	PASS only after a real API call; missing key/quota is BLOCKED.
3. Features work + deployed	Acceptance checks + public live URL smoke-tested.	Do not mark PASS before opening the deployed URL and running real flows.
4. Open-source repo + URL	Public GitHub repo, license, source, README, live URL.	Repo and URL must exist and be verified, not merely planned.
5. Innovation + real problem	Contribution Contract + Evidence-to-Test TraceMap + deterministic Claim Gate + copy-ready Maintainer Handoff + Contribution Proof Bundle + ProofBench.	Show before/after; report actual fixture results; no world-first claim.
6. Why open innovation matters	Explicit README section, docs/open-innovation.md and a PPT slide.	Explain inspectability, reuse, contribution, feedback and community adaptation in concrete terms.
7. Collaboration/community	CONTRIBUTING.md, Code of Conduct, issue templates, newcomer roadmap and a public ProofBench fixture contribution guide.	Artifacts must exist; do not claim people collaborated unless it happened.
8. Demo video or PPT	Editable submission/IssueLift_Submission.pptx with real screenshots and concise demo script.	Slides must match QA report; never use fake screens/results.
9. Setup guide	README setup plus .env.example and environment/key steps.	Verify commands from clean state; secrets remain absent.

WHY OPEN INNOVATION MATTERS — REQUIRED README + PPT TEXT

IssueLift turns an open issue into a reviewable Contribution Contract and evidence-to-test map before coding begins. The deterministic Claim Gate can be inspected and improved; the public ProofBench lets contributors add counterexamples whenever a path, citation, requirement or test-evidence label is wrong. This makes reliability rules—not only UI code—open to community inspection and extension. We claim no impact numbers until measured.

Community files: CONTRIBUTING.md, Code of Conduct, issue templates, docs/open-innovation.md, docs/newcomer-roadmap.md, docs/rubric-compliance.md, docs/competitive-audit.md, docs/eval-report.md, and the public

OPEN INNOVATION / COMMUNITY CONTRIBUTIONS

Make the project easier to improve.

ProofBench is designed for small, reviewable community contributions—not invented impact claims.

New contributors can help without changing the AI model.

Add a counterexample fixture, improve setup docs, or report an evidence-label bug.

01 **Add a ProofBench counterexample**

Create one small fixture for a false source ID, invented path, ambiguous criterion, or missing test evidence. State the expected invariant and why it matters.

02 **Improve a reproducible example**

Use a public, stable GitHub issue to improve a fixture or documentation example. Keep source links, observed output, and expected output separate.

03 **Fix a newcomer-facing document**

Clarify setup steps, environment-variable names, troubleshooting, or the meaning of VERIFIED / INFERRED / UNKNOWN. Do not add secrets or private data.

Contribution rule

A fixture benchmark validates listed guardrails only; it does not prove general model accuracy.
Never invent remote issues, contributor activity, impact numbers, or benchmark results.

What a good contribution includes

- A small, focused change with a clear explanation.
- Reproduction steps and expected vs. observed behaviour.
- Tests or fixture checks that were actually run, with the real result.
- No secrets, copied private data, fabricated sources, or unsupported claims.

Open innovation in practice

The rules, fixtures, and evidence labels stay inspectable so the community can challenge and improve them.

00C / FINAL AUTONOMOUS QA

Use after build; rerun after any late fix.

This is the release gate. It does not add major features; it proves what works and exposes what remains.

PROMPT 00C — FINAL QA, DEMO AND HANDOFF

```
Act as IssueLift final release engineer. Do not add major new features. Execute the acceptance flow:
1. Install from a clean state using README commands.
2. Run lint, typecheck, unit tests and production build.
3. Start the app and verify the health/readiness status.
4. Submit a real public GitHub issue URL and verify live issue metadata.
5. Verify repository context, missing-doc states and real source links.
6. If GEMMA_API_KEY is configured, call the actual Gemma 4 model and verify schema validation, Truth Ledger,
candidate paths and route. If absent, verify the missing-key screen and report live AI as BLOCKED.
7. Test open, closed, assigned, ambiguous and invalid issues; simulate API errors safely.
8. Verify Contribution Contract extraction/editing, Readiness Gate, deterministic Claim Gate, Contribution Route
Graph, Setup Truth, Maintainer Handoff, Acceptance-to-Test TraceMap/Test Evidence Radar and Contribution Proof
Bundle export. Verify Hint Ladder/related-issue signal only if implemented as P1; explicitly note any deferred P1
item.
9. Run `npm run eval:proofbench` (or documented equivalent) from a clean state; confirm fixture cases cover valid
evidence, ambiguity, missing test evidence, fabricated path/source ID, truncated context and untrusted issue text.
The report must distinguish fixture-guard tests from live model evaluation.
10. Confirm every visible button works and layout responds at phone/laptop widths.
11. Check git diff, build output, browser network responses and logs for secrets. Ensure .env files are ignored and
not tracked.
12. If deployment was authorized and completed, open the public URL and repeat key smoke tests. Otherwise report
deployment as PREPARED but NOT DEPLOYED.
13. Update docs/qa-report.md with exact commands, results, fixes, blocked checks and known limitations. Generate an
editable 6-8 slide submission/IssueLift_Submission.pptx deck from verified features and real screenshots; validate
that the file opens and its claims match the QA report.
14. Verify docs/rubric-compliance.md maps all nine event criteria to actual artifacts/status; ensure README
explicitly answers 'Why Does Open Innovation Matter?' and CONTRIBUTING.md, newcomer templates and community
playbook exist. Never mark a rubric item PASS without the corresponding artifact/evidence.

FINAL REPORT FORMAT
- Working local URL and command.
- Public deployment URL only if verified.
- GitHub repo URL only if created/pushed and verified.
- Checklist with PASS / FAIL / BLOCKED / NOT RUN.
- Files created/changed.
- Exact one next action for each external blocker.
- Confirm no billing/paid service was enabled.
- Confirm the actual PPTX deck exists, opens, and does not contain invented metrics or fake screenshots. Confirm
ProofBench results are from an executed command, not hard-coded marketing numbers.
Do not invent results. “Looks good” is not acceptance; only executed checks count.
```

UTILITY / RECOVERY

When Antigravity gets stuck or breaks something.

Paste this prompt only if the agent is looping, rewrites unrelated code or reports completion without proof.

DEBUGGING / RECOVERY PROMPT

```
Stop feature expansion and act as a senior debugging engineer for IssueLift.
1. Inspect the actual error, current file tree, changed files, terminal logs, browser console and network response.
2. Reproduce the failure using the smallest safe input.
3. Find the smallest root cause. Do not guess and do not rewrite unrelated modules.
4. Fix only the root cause and preserve working functionality, product requirements and API contracts.
5. Run the narrowest relevant test, then broader regression tests for the affected path.
6. Restart the app/build if appropriate and verify actual runtime behaviour.
7. If blocked by missing API key, quota, login or external service, finish everything else. Explain the exact blocker and the single smallest user action needed.
8. Never introduce paid services, fake production results, dummy pass states or hidden model fallbacks.
At the end report root cause, files changed, commands/tests executed, result and remaining limitations.
```

Emergency finish rule: if time is running out, stop feature expansion once live GitHub intake + Gemma analysis + source validation + Contribution Contract/TraceMap + Proof Bundle work. ProofBench’s minimal fixtures should still run; defer optional Hint Ladder/related issue signals first. Prefer an honest, tested smaller build over extra features that do not run.

USER ACTION / GEMMA API KEY

Do this only when Antigravity asks for the key.

No key is needed to begin the build. These steps are for the real Gemma integration test.

• 1. Open Google AI Studio. Visit aistudio.google.com/apikey and sign in using your Google account.

• 2. Open API Keys. If Google created a default project/key for you, it may already be visible. Otherwise click Create API key and follow the displayed steps.

• 3. Stay on the free tier. Do not click “Set up billing”, add a payment method, or activate a paid upgrade. Google lists Gemma 4 input/output as free of charge on its current Gemini API pricing page, but model access and quotas still apply. If the page requires billing for your account, stop and tell Antigravity; do not enable it.

• 4. Copy the key privately. Treat it like a password. Do not paste it into a chat message, GitHub issue, README, screenshot or public repository.

• 5. Put the key in the correct local file. In the Antigravity file explorer, look for `.env.local` in the IssueLift project root. If it does not exist, create a new file named exactly `.env.local`; copy the line format from `.env.local.example` if that template exists. Edit `.env.local` only: replace the placeholder after `GEMMA_API_KEY=` with your key and save. Never put the real key in `.env.local.example` or `.env.example`. Keep `.env.local` ignored by Git.

• 6. Ask Antigravity to verify it. In chat, say only: “I added the key to the ignored `.env.local` file. Please run the real Gemma connection test without printing the key, and confirm `.env.local` is not tracked by Git.” Never send the key value in chat.

• 7. If deploying to Vercel. Open the Vercel project → Settings → Environment Variables. Add `GEMMA_API_KEY` and the secret value for the relevant deployment environments. Redeploy after saving. Do not add it to any variable whose name starts with `NEXT_PUBLIC_`.

• 8. If you see quota/API errors. Do not enable billing. Ask Antigravity to inspect the exact error, show a clear retry/quota state, and leave the AI call blocked until free quota is available.

Privacy note: Google’s published free-tier terms state that submitted content may be used to improve its products. This MVP sends public repository/issue content only. Do not scan private repositories, private source code or secrets.

USER ACTION / PUBLIC DEPLOYMENT

Antigravity prepares; you authorize the account connection.

Only do this after local tests pass and the app works with Gemma configured.

• 1. Create a public GitHub repository. Create an empty repository named `issuelift` under your GitHub account. Do not upload any `.env` file.

• 2. Ask Antigravity to prepare a clean commit. It should check `git status`, verify secrets are ignored, add README/license/docs and only then ask before pushing the code.

• 3. Open Vercel. Sign in at vercel.com using GitHub. Choose the Hobby/free plan for this personal hackathon demo.

• 4. Import the `issuelift` repository. Vercel should detect Next.js automatically. Use the default build settings unless Antigravity documents a specific required adjustment.

• 5. Add environment variables. Add `GEMMA_API_KEY` in Vercel project settings. Never commit it to the repo. If the app uses `GEMMA_MODEL`, set it to the verified model ID configured in the README.

• 6. Deploy and verify. After deployment completes, open the generated Vercel URL. Run one real issue scan, check an error path, and confirm the key never appears in the browser network response or source bundle.

• 7. Report it honestly. If login or deployment is blocked, keep the app functional locally and ask Antigravity for the exact missing action. Do not claim the site is live until it has been opened and tested.

Before submitting	What to verify
Public repo	README, license, setup guide, architecture and no secrets.
Live app	Real issue fetch and Gemma analysis with configured key.
Report integrity	Verified / inferred / unknown; source links match fetched data.
Failure states	Missing key, invalid URL, GitHub limit and Gemma quota all handled honestly.
PPT/demo	Screenshots from the actual running app; say what has and has not been verified.

NON-NEGOTIABLE / ZERO-BUDGET GUARDRAILS

No surprise charges. No fake success.

The free plans are designed for a prototype; quotas and policies can change.

Component	Allowed	Never do
Antigravity	Individual $0 plan; use available basic weekly quota.	Never ask the user to buy Pro/Ultra just to finish routine work.
Gemma 4	Gemini API lists Gemma 4 input/output as free of charge on the current pricing page; quota and account/model access still apply.	Never enable billing, choose paid model/services, or silently switch to a paid endpoint.
GitHub API	Public repo endpoints; cache and limit calls.	Never request a token unnecessarily or brute-force API calls.
Hosting	Vercel Hobby if eligible; no custom domain.	Never add paid integrations or enable paid features.
Secrets	`.env.local` locally; host environment secrets in deployment.	Never commit, expose in browser, log or include in screenshots/PPT.

Official references checked 25 October 2026

Antigravity pricing / individual plan — $0/month with basic weekly rate limits
https://www.antigravity.google/pricing

Gemma 4 on Gemini API — Official supported model IDs and SDK examples
https://ai.google.dev/gemma/docs/core/gemma_on_gemini_api

Gemini API keys — Create/manage API keys; secure environment setup
https://ai.google.dev/gemini-api/docs/api-key

Gemini API pricing — Free/paid tier terms and quota caveats
https://ai.google.dev/gemini-api/docs/pricing

GitHub REST API rate limits — Public unauthenticated rate-limit behaviour
https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api

Vercel Hobby — Hosting plan details and limits
https://vercel.com/pricing

Definition of done: a live public issue can be scanned; current GitHub data is displayed; Gemma produces structured analysis when its key/quota is available; the Contribution Contract distinguishes explicit/inferred/unknown requirements; TraceMap maps criteria to source/test evidence or bounded-scan gaps; the deterministic Claim Gate rejects fabricated sources/paths; Maintainer Handoff and Contribution Proof Bundle work; ProofBench runs and reports real fixture results; required open-innovation artifacts exist; the nine-criterion rubric matrix reflects actual checks; no secrets or paid dependencies are added.

This pack intentionally leaves room for a final human review. An autonomous coding agent can implement and verify code, but it cannot guarantee external account access, free quota availability, a successful deployment, or a hackathon prize.